# An Embedding-Based Retrieval Pipeline for Equation Formulas in Scientific Papers

## 1 Overview

The task is to extract numbered equations from quantum physics arXiv papers and produce, for each equation, a meaning sentence, symbol definitions, inter-equation relations, and an audit trail. Because the required dataset must be traceable to the paper, generated explanations are avoided.

The approach is a hybrid retrieval system inspired by RAG. Instead of dynamic user queries, the project has three fixed query types: equation meaning, symbol meaning, and equation relation. The pipeline parses arXiv HTML into equations, text chunks, context sentences, and MathML symbols, then embeds equations and chunks with **MathBERT** [1] (`witiko/mathberta`). **MathBERT** is used because it is trained for mathematical and scientific language, so it can compare equations with explanatory text better than a general text embedding model. BM25 and **MathBERT** provide model-based retrieval, while deterministic rules choose only source-backed spans. This keeps the method simple, general across papers, and auditable.

## 1.1 Pipeline Diagram

The diagram below shows the data flow through the ten stages.

```
 [arXiv]
    |
    v
Stage 1: Download HTML
    |  data/html/<paper_id>.html
    v
Stage 2: Parse Documents
    |  data/pipeline/documents/<paper_id>.json
    |  (sections, paragraphs, sentences, MathML)
    v
Stage 3: Build Chunks
    |  data/pipeline/chunks/<paper_id>.json
    |  (sentence | paragraph | neighborhood | cross_reference)
    v
Stage 4: Extract Equations
    |  data/pipeline/equations/<paper_id>.json
    |  (reviewed LaTeX, raw equation id, anchor_id,
    |   section, before/after sentences, audit context)
    v
Stage 5: Build MathBERT Embeddings
    |  data/pipeline/embeddings/<paper_id>.{npz,json}
    |  Family A: equation / summary / before_sentence / after_sentence
    |  Family B: sentence_chunk (whole paper)
    |
    +------------------+------------------+------------------+
    |                  |                  |                  |
    v                  v                  v                  v
Stage 6:           Stage 7:           Stage 8:           Stage 9:
Equation           Symbol             Symbol             Relations
Meanings           Extraction         Definitions        Detection
(cue/xref/BM25     (MathML tree       (embed pre-        (cue phrases,
 + cosine rank)     traversal)        filter + patterns)   semantic sim.)
    |                  |                  |                  |
    v                  v                  v                  v
data/pipeline/meanings/ data/pipeline/symbols/  data/pipeline/symbol_meanings/ data/pipeline/relations/
<paper_id>.json    <paper_id>.json    <paper_id>.json    <paper_id>.json
    |                  |                  |                  |
    +------------------+------------------+------------------+
                       |
                       v
                  Stage 10: Export
                       |  join fields and add audit-trail
                       |
                       v
              data/final_data.json
```

## 2 Building Embedding Space

This section explains how the pipeline prepares the shared retrieval space used by the later extraction stages. Stages 2–4 convert each arXiv HTML paper into structured inputs for embedding:

- **Stage 2: document parsing** reads the cached HTML and stores sections, paragraphs, sentences, raw equation blocks, labels, MathML, cross-references, and local before/after context in data/pipeline/documents/<paper_id>.json.
- **Stage 3: chunk construction** turns the structured document into searchable units, including sentence chunks, paragraph chunks, equation-neighbourhood chunks, and cross-reference chunks in data/pipeline/chunks/<paper_id>.json.
- **Stage 4: equation extraction** selects the numbered display equations for the dataset and stores their equation ID, LaTeX, HTML anchor, section, document order, and resolved context sentences in data/pipeline/equations/<paper_id>.json.

Stage 5 is the key step that turns these structured objects into an embedding space.

**Responsible file:** Ourapproach/stage05_build_embeddings.py
**Output:** data/pipeline/embeddings/<paper_id>.json
and data/pipeline/embeddings/<paper_id>.npz.

This stage embeds equations and text chunks with **MathBERT**. For each equation it builds equation-only, summary-context, and sentence-context vectors. For the paper text it embeds the retrieval chunks from Stage 3. The vectors are saved in .npz; the JSON file records row metadata and source text for auditing.


```
paper_id: 2401.00059
model: witiko/mathberta
normalized: true
dimension: 768
rows:
  vector_id: 2401.00059:equation:1:summary
  vector_kind: summary
  source_text: before text, equation text, after text
  row: 1
```


## 3 Retrieval and Extraction

**Equation meanings.** After Stage 5, the pipeline uses the same embedding space for all final annotations. Equation meanings are selected from candidate sentences collected by cross-references, cue phrases, BM25 retrieval, and local proximity. Candidates are ranked by

$$\mathrm{score}(c) = 0.5\cos(e,c) + 0.2\,\mathrm{BM25}(e,c) + 0.2\,\mathrm{source}(c) + 0.1\,\mathrm{prox}(e,c) - \mathrm{penalty}(c).$$

The terms are:

- $\cos(e,c)$: **MathBERT** similarity between equation context and candidate sentence;
- $\mathrm{BM25}(e,c)$: lexical overlap with the equation query;
- $\mathrm{source}(c)$: bonus for reliable evidence such as cross-references or cue phrases;
- $\mathrm{prox}(e,c)$: preference for text near the equation;
- $\mathrm{penalty}(c)$: penalty for procedural, incomplete, math-heavy, or symbol-definition-like sentences.

Stage 6b then applies a rule-based postprocessing step in `stage06_postprocess_meanings.py`. The rules shorten the top candidate sentence into an extractive meaningful phrase.

```
Before postprocessing:
The covariance matrix of the TMSV state can be obtained by setting
n_{\\text{th}}=0 in Eq. (1).

After postprocessing:
covariance matrix of the TMST state
```

**Symbol definitions.** Symbols are extracted mainly from MathML because it preserves subscripts, superscripts, and decorated notation; a LaTeX fallback is used when MathML is incomplete. For each symbol, the system builds aliases from its canonical form, base symbol, Unicode form, and LaTeX spelling. These aliases are combined with definition phrases such as "where $x$ is" or "$x$ denotes" and embedded as a query. The top retrieved sentences are checked with deterministic patterns, including where-is, denotes, let-be, reverse noun phrase, and respectively-list patterns. This two-step design keeps retrieval flexible while making the final definition conservative. If no reliable source span is found, the definition is left empty rather than guessed.

**Relations.** Relations are built for every ordered equation pair in the same paper. The direction matters: relation $(A, B)$ asks whether the context around equation $A$ refers to or depends on equation $B$. A relation is marked *strong* when the paper gives direct evidence, such as a citation with derivation, equivalence, or special-case cues, a plain explicit citation, or membership in the same detected equation block. It is marked *potential* when weaker evidence exists, such as shared symbols, high **MathBERT** similarity between equation summaries, or section proximity. If no signal fires, the relation is *none*. Thus strong edges are evidence-backed claims, while potential edges are graph candidates that may need manual interpretation.

**Export.** Stage 10 finally joins meanings, symbols, definitions, relations, and audit fields into the required JSON.

# 4 Conclusion

### Strengths.

Our approach builds the required dataset by combining structured arXiv parsing, embedding-based retrieval, deterministic patterns, and audit validation. Its main strengths are traceability, graph-ready coverage, and flexibility across quantum physics writing styles. The hybrid design also avoids hallucination: **MathBERT** finds relevant evidence, but rule-based checks decide what can enter the final JSON.

### Weaknesses.

Symbol definitions remain incomplete when papers introduce notation implicitly, and potential relation edges can be broad when they rely only on weak contextual signals. These limits are difficult to remove without generating unsupported explanations, which would violate the extractive-only requirement.

### Possible improvements.

Future work could fine-tune **MathBERT** on equation-context pairs from quantum physics papers, calibrate semantic thresholds for relation detection, and add more notation-specific definition patterns.

A small manually checked validation set would also help tune the balance between coverage and precision.

## References


[1] S. Peng, K. Yuan, L. Gao, and Z. Tang, *MathBERT: A Pre-trained Model for Mathematical Formula Understanding*, arXiv preprint arXiv:2105.00377, 2021.